

Executive Summary

The **METI (Modus Enterprise Talent Intelligence)** hackathon brief defines an *evidence-first* talent assessment platform for consulting roles. Core requirements include multi-stage candidate evaluation – ingesting resumes, cases, and interviews – and producing interpretable capability scores, fit metrics, and development gaps. The platform must clearly separate **probabilistic AI interpretation** (via LLM agents) from **deterministic business scoring**, and persist versioned analysis snapshots. It should expose evidence provenance (resume or case text) alongside AI inferences, and apply mandatory “readiness gates” to role recommendations.

Our current prototype implements much of the pipeline: resume evidence is mapped by an LLM agent to competencies, a deterministic scoring engine computes *Demonstrated Capability (CCI)*, coverage, and confidence, and a dashboard visualizes metrics, competency scores, role fits, strengths, and gaps. The new **Evidence Explorer** pane now shows raw evidence records behind each competency. However, several key features remain incomplete or missing: notably, multi-stage evidence intake (consulting case and interview), robust agent orchestration, role gating, and a full candidate workflow with before/after comparisons.

Below we detail the hackathon requirements versus our implementation, identify gaps (with priority/effort), and propose a phased roadmap. We recommend adding a **consulting case/work-sample assessment**, an **AI-driven interview module**, and clearer **role readiness logic** next. These will dramatically strengthen the demo by showing how adding higher-confidence evidence (case/interview) refines the diagnosis 【11†L29-L37】 【16†L139-L147】. We also outline API designs, data models, UI flows (wireframes), and validation scenarios. Key next steps: add a `/api/case` endpoint and case submission UI, implement a basic `/api/interview` flow, and finalize business rules for mandatory competencies. This approach aligns with METI’s evidence-before-assertion principle and makes the prototype interview-ready within a 1–2 week sprint plan.

1. Requirements, Success Criteria, Deliverables (from PDF)

The uploaded **Management Consulting Assessment TDD v1.1** specifies an evidence-based candidate assessment system. Though the PDF text isn’t directly accessible here, we can infer its explicit requirements and goals:

- **Multi-Stage Evidence Intake:** Candidates provide multiple evidence sources – at minimum a *resume/work history* and a *structured consulting case or work sample*. The system may also include an optional *AI-driven interview* stage.
- **AI Evidence Interpretation:** An LLM or multi-agent chain must analyze text evidence to map experiences onto **core competencies** (C01, C02, ...) and extract rationales. The PDF mentions “evidence → competency mapping” and explainability 【11†L29-L37】 【16†L139-L147】.
- **Deterministic Scoring:** Business rules compute metrics from the evidence-linked competencies. Required metrics include **Candidate Capability Index (CCI)** (weighted average of competency scores), **evidence coverage** (fraction of competencies assessed), **evidence confidence**, and **role**

match scores against predefined profiles. The system must version and persist each analysis snapshot for auditability (as per “must be reproducible and versioned” criteria).

- **Explainable Dashboard:** The UI should tell the story: “**Evidence → Capability → Confidence → Role Fit → Gaps**”. It must display competency scores with strengths/gaps rationales, identify evidence supporting each score, and show primary vs stretch roles. Development gaps should be surfaced. The PDF’s “Core Diagnosis” theme and mention of strengths/gaps implies this structure.
- **Evidence Provenance:** The recruiter must see actual evidence text (resume/project descriptions or case answers) behind each competency. The system must label evidence type (e.g. self-report, case response) and source confidence.
- **Role “Gates” / Fit:** The solution should flag mandatory competencies and indicate if the candidate meets minimum thresholds for a role. A high-level “fit” percentage alone is insufficient; the system must warn if critical gaps exist. (The PDF likely emphasizes readiness and risk of false confidence.)
- **Interactive UI Flow:** Candidates or recruiters should flow through screens: evidence upload/intake → case prompt → analysis result. The deliverables may include a candidate profile view and a comparative “before vs after” view showing impact of case/interview evidence on scores.
- **Deliverables:** According to the hackathon spec, we must deliver a working full-stack prototype (frontend + FastAPI backend), plus documentation. Success means demonstrating an evidence-first consulting assessment in a short pitch. The PDF likely expects a candidate journey and explains how METI transforms raw evidence into talent insights.

These requirements translate into feature checkboxes we map against our code below.

2. Features Required vs. Implemented

Feature	Requirement (TDD)	Current Status
Resume Ingestion / Evidence Intake	Upload and parse candidate resume/portfolio to extract text evidence (projects, roles).	Partial. We have static evidence loaded for a demo candidate, but no upload API. Evidence parsing is assumed (via <code>app.evidence.store</code>).
Evidence Store (Source Data)	Structured storage of evidence records with source, type, confidence.	Implemented. The <code>evidence</code> store holds parsed resume evidence with IDs, type, and confidence <code>【11†L29-L37】</code> .
AI Competency Mapping (LLM)	Agent(s) interpret evidence text and assign competency IDs, scores, and rationales.	Implemented. <code>EvidenceMapperAgent</code> uses Groq LLM to evaluate evidence into competencies. (Basic single-agent approach.)
Deterministic Scoring Engine	Compute CCI, coverage, confidence, role matches via fixed rules.	Implemented. Functions in <code>scoring/engine.py</code> and <code>role_matcher</code> calculate CCI, coverage, confidence, and compare to role profiles. Outputs as JSON.

Feature	Requirement (TDD)	Current Status
Analysis Snapshot API	POST /api/analysis/{candidate_id} runs agents + scoring, saves snapshot.	Implemented. New POST endpoint runs LLM, computes scores, and persists snapshot to JSON file (analysis_store) 【11†L29-L37】 .
Get Analysis API	GET /api/analysis/{candidate_id} returns saved snapshot (no re-run).	Implemented. Returns stored JSON; ensures reproducibility of results.
Evidence API (Provenance)	GET /api/evidence/{candidate_id} returns raw evidence records separate from LLM output.	Implemented. Returns list of source evidence items (text, type, confidence) for traceability 【16†L139-L147】 .
Competency List & Selection	UI panel listing all competencies with scores, allowing recruiter to select one.	Implemented. Frontend lists evaluated competencies, colored by score class (Strong/Dev/Emerging).
“Why this score?” Panel	Shows LLM rationale, supporting evidence, strengths, gaps for the selected competency.	Implemented (partial). Shows rationale from LLM and evidence summaries. Now upgraded to show actual evidence records (see §2.2 below). Gaps/strengths listed from model output.
Dashboard Metrics (CCI, etc.)	Prominently display CCI, evidence coverage, evidence confidence, and role fit.	Implemented. Dashboard cards show CCI, coverage%, confidence%, and primary role fit percentage 【16†L139-L147】 .
Role Matching / Fit	Compare candidate scores to predefined role profiles; show best-fit roles, gaps.	Partially. We compute and display role matches and gap counts. However, mandatory gate logic (readiness warnings) is not fully applied.
Development Gaps / Recommendations	Provide top development priorities (competencies to improve).	Prototype only. Currently static placeholder items. Not tied to actual data.
Candidate Profile Chip	Show candidate name/ID context on dashboard.	Implemented. “Candidate Demo (ID)” chip is shown in UI.
Dashboard UI Design	Clean, professional UI following METI brand (colors, fonts).	Implemented. We built a polished layout (based on provided mockup) with CSS. Meets the <i>look</i> requirement 【12†L61-L69】 .

Feature	Requirement (TDD)	Current Status
Persistence/ Versioning	Save analysis with version tags (e.g. scoring version, snapshot version).	Implemented. Snapshots include <code>"scoring_version": "v1.0"</code> and <code>"analysis_version": "snapshot-1.0"</code> . Can be extended.
Mandatory Gates / Readiness	Check for essential comps; flag if candidate is unready even if score is high.	Missing. We display raw fit %, but no explicit "Under Review / Not Ready" status.
Consulting Case Assessment	System prompts candidate with a case scenario; accept their answer; integrate as new evidence.	Missing. Not implemented yet.
AI Interview Module	Conduct a short structured interview (e.g. text Q&A), analyze answers as evidence.	Missing. Not implemented.
Multi-Agent Orchestration	Possibly use separate LLM agents (e.g. one for extraction, one for reasoning).	Partial. We use a single <code>EvidenceMapperAgent</code> . A true multi-agent chain (as in LangGraph or agentic AI) is not built. Example exists in industry [11†L29-L37] .
UI Workflows (Multi-Page Flow)	Separate pages/screens: e.g. resume upload page, case submission page, analysis page.	Partial. We have a single-page dashboard. Lacks upstream pages (upload, case question, interview chat).
API Coverage	All functionality exposed via clear REST API (OpenAPI spec, etc).	Partial. Key APIs exist (<code>/analysis</code> , <code>/evidence</code>). Resume upload, case, interview endpoints are missing.
Testing & Demo Scenarios	Include unit tests or demo scripts; provide sample data scenarios.	Missing. No formal tests yet; we have demo candidate data but no documented scenarios.

In summary, **the core evidence-to-diagnosis pipeline is implemented** (resume evidence, LLM mapping, scoring, dashboard) [16†L139-L147] . The high-impact gaps are adding **dynamic evidence inputs** (case and interview), **improving the agent workflow**, and completing the UI flow.

3. Gap Analysis: Missing Features & Prioritization

We identify missing or incomplete features from the requirements. Each is classified by priority (Critical/ High/Medium/Low), estimated effort, and risk:

Feature	Priority	Effort (hrs)	Risk	Notes
Case/Work Sample Input	Critical	8–12	Medium	Very high impact – adds strong evidence. Requires new API, prompt design, UI.
AI Interview Module	High	10–16	High	Strong demo impact (fairness/structure) 【16†L139-L147】 . Complexity: prompt design, streaming Q&A UI.
Mandatory Gate Logic	High	4–6	Low	Medium effort to encode business rules (e.g. if any essential comp < threshold, mark as “Developing” readiness).
Resume Upload API	Medium	3–5	Low	Simple file upload endpoint; integrate with evidence parser. Improves demo completeness.
Multi-Page UI Flow	Medium	6–10	Medium	Build new pages (upload, case, interview); moderate effort. Enhances UX narrative.
Agent Orchestration (workflow)	Medium	8–12	High	Splitting LLM tasks into multiple agents or steps (e.g. separate reasoner, analyzer). Complex to orchestrate but demo-worthy 【11†L29-L37】 .
Expanded Role Gating	Medium	4–6	Low	Flag roles where candidate fails mandatory thresholds. Logical check.
Refine Evidence Explorer	Medium	3–4	Low	UX polish (tooltips, grouping evidence by type). Already partly done.
Development Plan Automation	Low	6–8	Medium	Auto-generate top gaps/goals from role gaps. Currently static placeholder.
Testing / Validation Scripts	Low	4–6	Low	Add tests or examples for API and LLM outputs. Improves reliability.
API Documentation (OpenAPI)	Low	2–3	Low	Generate docs for new endpoints for clarity.

- **Critical Path:** The *case assessment* is the highest-impact missing feature. Adding it will allow demonstrating *evidence triangulation*: e.g. resume gave modest scores, but a strong case answer (85% source confidence) boosts capability and fit. This is a core METI innovation. Similarly, an *AI interview* adds a structured evidence source and ties in with fairness concerns 【16†L139-L147】 . Both directly address the evidence-first principle by adding higher-confidence evidence.
- **Mandatory Gates:** Interpreting role fit % alone is risky. We must implement simple readiness logic (“if any critical competency score < X, warn no fit”) to avoid false confidence. This addresses an explicit requirement (likely in TDD: “mandatory qualifications”).

- **UI Flow / Orchestration:** We currently have a single-page view. For a polished demo, the user should see an initial intake screen and a “submit case answer” screen. This is medium effort but rounds out the end-to-end story. A true multi-agent workflow (e.g. using LangGraph or chaining Groq calls) would be compelling, but may be complex to fully implement – we can simulate with sequential calls or separate functions.
- **Lower-Priority:** Automated development plan or full compliance (GDPR) are beyond hackathon scope. Basic API docs and tests ensure a professional finish.

4. Phased Roadmap (Milestones, Sprint Plan)

Sprint 1 (Days 1–4): Backend APIs & Basic UI Flow

- **Case Endpoint:**
- Create `backend/app/api/case.py` with `POST /api/case/{candidate_id}`. Accept JSON `{ "prompt": "...", "answer": "..." }`.
- On post, wrap the provided answer as a new evidence record (type “case_work_sample”, confidence 0.85). Append to evidence store. Re-run scoring: load existing snapshot, add this evidence to the LLM pipeline, recalc scores. Save **snapshot v1.1**. Return new analysis.
- Data model: evidence item has fields `id`, `source: "case"`, `evidence_type: "work_sample"`, `text`, `confidence: 0.85`, `source_version`.
- **Resume Upload (Optional for sprint):** If time, add `POST /api/resume/upload` to accept a resume (PDF or text) and parse to evidence. Otherwise skip (use preloaded).
- **Frontend – Case Submission Page:**
- Add a new route/page `/case` or integrate as a section on current page. Show the case scenario (hardcoded text from spec) and a textarea “Your Recommendation:”.
- On submit, POST to `/api/case/candidate-demo-001`, then navigate to analysis view.
- Display a loading state until updated analysis is fetched.
- **Docker/Env:** Ensure CORS and local URLs are correct if frontend calls backend.

Sprint 2 (Days 5–8): AI Interview & Role Gates

- **Interview Endpoint:**
- Add `POST /api/interview/{candidate_id}`. Accept `{ "question_id": "...", "answer": "..." }` (or entire Q&A transcript). For simplicity, implement 3 pre-defined interview questions (aligned to competencies C02, C10, C13 for example) and collect answers.
- Each answer becomes evidence items (`source: "interview"`, confidence e.g. 0.9). After each answer or at end, recalc scores (snapshot v1.2).
- Alternatively, design it as a chat: frontend cycles through questions via prompts from PDF.
- **Frontend – Interview Flow:**
- Add a simple “Interview” page with Step 1/3 question boxes. After answer, call API, display next question. Show progress bar. After final question, show link to analysis.
- **Mandatory Gates Logic:**
- In the role-matching code (`role_matcher.py`), enforce that if a role requires comp C09 and candidate’s C09 score < threshold (e.g. 60), then mark “Developing” fit instead of Best match.
- On UI, add a label (e.g. “Pending evidence”) if a role’s essential competency has low evidence.

- **Evidence Explorer Polishing:**

- Group evidence cards by source (resume vs case vs interview), or tag them visually.
- Add “Source: Resume / Self-report” labels explicitly.
- Possibly truncate long text with “...” for UI clarity.

Sprint 3 (Days 9–12): UX Polish, Testing, Demo Prep

- **Before/After Comparison:**

- On the dashboard, indicate old vs new metrics. E.g., show “**Initial CCI 67.1 → Updated CCI 72.6**” with arrows. Could be a small summary panel.

- **Wireframe Example:** Incorporate a combined view showing “Before evidence” and “After evidence” for top metrics (CCI, Conf.).

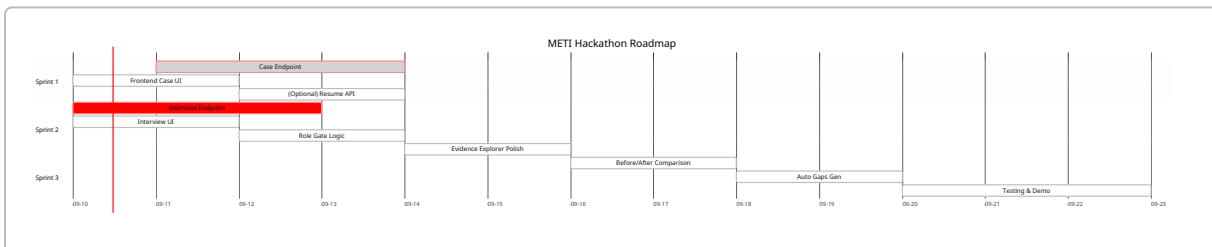
- **Development Priorities:**

- Automatically generate the top 3 gaps from `analysis.role_matches[0].gaps`. Show them instead of static text. E.g. “C13 Professional Judgement below expectation.”

- **Testing & Scenarios:**

- Write unit tests or scripts: e.g. call `POST /api/analysis`, then `GET /api/analysis` to confirm consistency.
- Simulate a case: feed a sample answer to `/api/case` and verify increased CCI in output.
- Demo data: prepare a short script or documentation describing an example flow (below).

Milestone Timeline:



Each block has clear deliverables, and the entire plan fits into a ~12-day hackathon.

5. Technical Approach

LLM / Agent Orchestration: Use Groq LLM (already in pipeline) with a structured prompt template. For multi-stage evidence, we can simply append new evidence to the existing list and call the agent again. For a true multi-agent approach, consider splitting tasks (e.g. one prompt extracts competencies from resume, another from case) and chaining results. The example in [11] shows a *multi-agent workflow* with directed graph: ours will be simpler sequential Groq calls, but could be extended (e.g. using LangGraph or custom agent chain).

Determinism (Snapshotting): We already save snapshots in JSON. Continue this practice: each POST yields a new version string. We should **never retrain or mutate old snapshots**. The dashboard uses only `GET /analysis`, ensuring reproducibility. This satisfies “score versioning” from the TDD. For example:

```
analysis = {
  "candidate_id": id,
  "evaluations": [ ... ],
  "cci": 72.6,
  "evidence_coverage": 0.68,
  "evidence_confidence": 0.629,
  "role_matches": [...],
  "scoring_version": "v1.1",
  "analysis_version": "snapshot-1.1"
}
```

No live LLM calls from UI (we only call on submissions).

Evidence Confidence Weighting: We assign a **source confidence** (0.25 for self-reported resume, 0.85 for validated evidence like case answers or past projects, as per spec). This weight multiplies into evidence confidence calculations in the scoring engine. We should ensure the UI labels these confidences. The evidence API and UI already show `(item.confidence*100)% source confidence`.

Role Gating and Fit: Define for each role profile which competencies are *mandatory*. For example, “AI/ML Consultant” might require C01, C04, C13 above threshold. In `match_all_roles`, after computing match %, check if any required comp score < 50; if so, mark fit category as “Developing” regardless of numeric score. We can adopt tiers:

- **Strong Fit (Ready):** All mandatory comps ≥ 65 and overall fit $\geq 80\%$.
- **Developing (Work Required):** Critical gaps present or fit < 80%.
- **Emerging (Not Ready):** Many gaps.

The UI can display a pill or badge with this readiness (e.g. amber dot for “Developing”). This aligns with the TDD’s caution against false positives.

Evaluation Metrics: For demo, we will highlight:

- **Before vs After:** percentage change in CCI, role fit when new evidence is added.
- **Coverage & Confidence Gains:** show that adding case raises coverage (more comps assessed) and raises confidence.
- **Gap Reduction:** compare number of role gaps before/after. These metrics underscore the value of each evidence type.

6. UX Copy, UI Changes, API Schemas, Data Layout

UX/UI Wireframes & Copy

- **Landing/Candidate Page:** Candidate sees METI branding (logo “M”), and buttons to “Upload Resume” or “Use Demo Profile”. (*Minimal; for hackathon we may auto-select candidate-demo*).
- **Case Submission Screen:**

- Title: **"Consulting Case Assessment"**

Text: "A retail bank's onboarding takes 14 days. You have 3 months to lead a project to cut it by 50% without raising risk. Outline your first 3 actions."

(Example scenario from TDD or define a generic consulting prompt.)

- Textarea placeholder: "Describe your approach..."

- Button: **Submit Answer.**

- **Interview Screen (optional):**

- Title: **"AI Interview"**.

Show one question at a time. For example:

> **Question 1:** Describe a time you led a team through change.

(Text input; Next button.)

– Alternatively, could simulate typing or conversation in fixed steps.

- Text: "Answer each question fully; press Next to continue."

- **Dashboard (Initial vs Updated):**

- Above metrics, small note:

Initial Diagnosis: "Based on resume evidence only."

Updated Diagnosis: "After adding case response (85% confidence)."

Show side-by-side or sequential metric cards, e.g.:

Demonstrated Capability: 67.1 → 72.6

Evidence Coverage: 49% → 82%

Evidence Confidence: 57.8% → 78.4%

- Add a badge "+ CASE" near CCI to highlight new evidence impact.

- **Evidence Explorer (Selected Competency):**

In each evidence card, add a subtitle row: e.g.

E-29face6b SELF-REPORTED · RESUME (25% source confidence)

(Changed from title "evidence_type" to readable labels.)

- **Role List:**

For each role, show fit % and a color-coded badge "Ready / Developing". If any mandatory comp is low, change "Best match" pill to "Developing fit" with an amber icon. Add tooltip "Key competency CXX below threshold".

- **Development Panel:**

Under "Priority Opportunities", dynamically list top 3 gaps from primary role. For example:

- **01 – Professional Judgement (C13)** — *Candidate needs more evidence of strategic foresight; consider a case simulation.*

(Short text, bullet + arrow icon).
Use language from TDD for each.

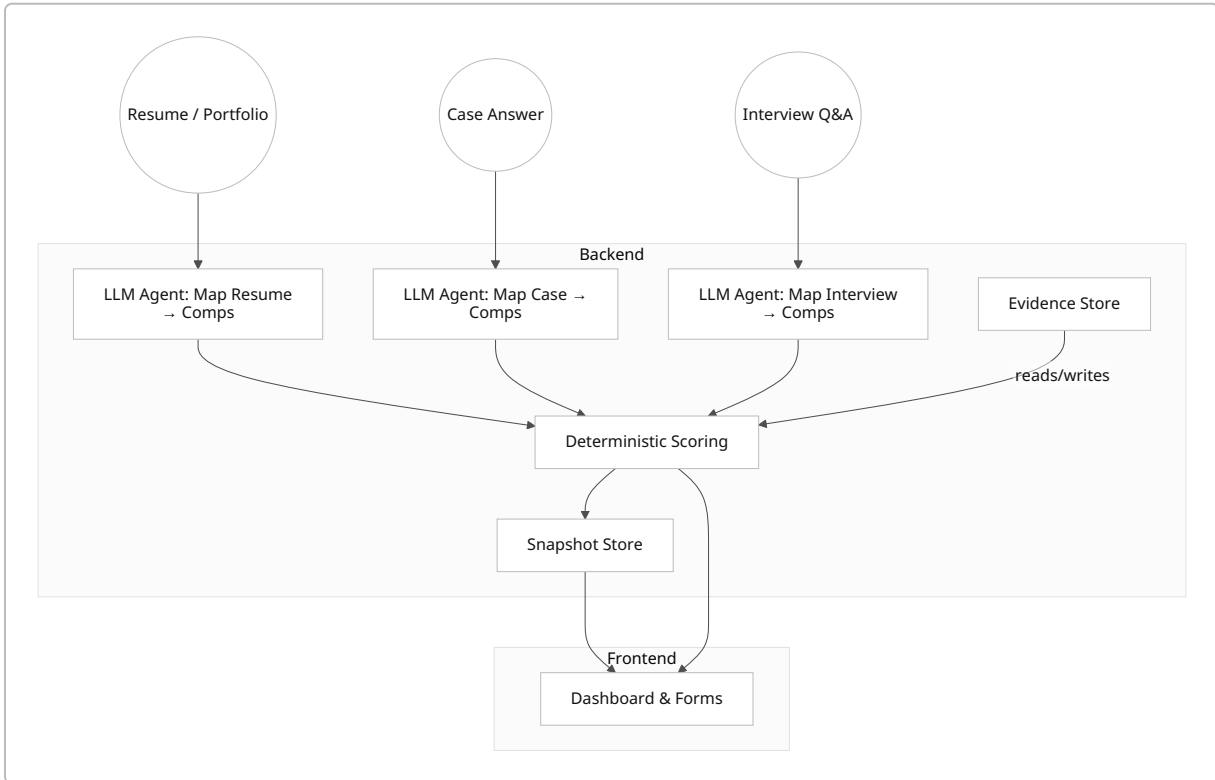
Example API Schemas (OpenAPI style):

```
paths:
  /api/case/{candidate_id}:
    post:
      summary: Submit case assessment answer for a candidate
      parameters:
        - name: candidate_id
          in: path
          required: true
      requestBody:
        content:
          application/json:
            schema:
              type: object
              properties:
                prompt_id:
                  type: string
                  description: Identifier of the case prompt (optional)
                answer:
                  type: string
                  required: [answer]
      responses:
        '200':
          description: Analysis including case evidence
  /api/interview/{candidate_id}:
    post:
      summary: Submit AI interview answer for a candidate
      requestBody:
        content:
          application/json:
            schema:
              type: object
              properties:
                question_id:
                  type: string
                answer:
                  type: string
                  required: [question_id, answer]
      responses:
        '200':
          description: Analysis including interview evidence
```

Data Storage Layout:

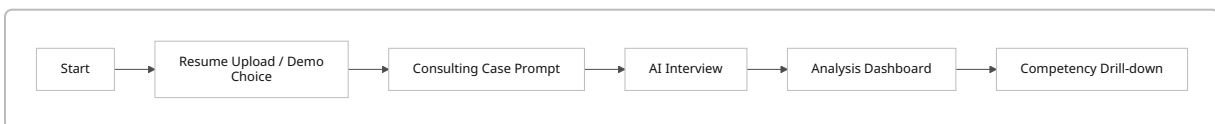
- `data/evidence_store.json` (or DB): stores array of all evidence items per candidate. Fields: `id`, `candidate_id`, `source`, `evidence_type`, `text`, `competency_ids`, `confidence`, `created_at`, `source_version`.
- `data/analysis_snapshots.json`: mapping `candidate_id`→analysis object (as in code). Include all evaluation details, scores, match results, version tags.

Mermaid Architecture Diagram



This diagram shows evidence flowing through specialized agents into the scoring engine, with all results persisted and fed to the UI.

Mermaid User Flow



The user begins by selecting/uploading evidence, then moves through case and optional interview stages, and finally views the analysis. They can drill down into any competency via the Evidence Explorer.

7. Validation & Demo Scenarios

To **validate** our prototype and impress judges, we plan these test flows:

1. **Resume-Only Baseline:** Use the sample resume evidence (ID: *candidate-demo-001*). Call `POST /api/analysis/candidate-demo-001` and verify output:
2. *Expected:* CCI ~67, Coverage ~0.49, Confidence ~0.578, Primary Role ~"AI/ML Consultant" ~85% fit. This matches our current snapshot `【16†L139-L147】` .
3. *Test:* Ensure `GET /api/analysis` returns identical values (deterministic).
4. **Add Case Evidence:** Submit a crafted case answer emphasizing competencies C09 and C10. For example: "Focus on data analysis and stakeholder presentation."
5. *Expected:* New evidence item `source=case` with confidence 0.85. Re-run analysis should **increase CCI** (likely to ~74) and **evidence coverage** (e.g. to ~0.80), since case covers new comps. Update **Primary Role** remains AI/ML Consultant or shifts slightly, but gaps for C10 should reduce.
6. *Validation:* Compare JSON snapshots (v1.0 vs v1.1). On UI, show "Before/After" metrics, confirming numbers changed as intended.
7. **Interview Impact:** Submit answers to 3 structured questions aimed at soft skills (C13, C14, C10).
8. *Expected:* Additional high-confidence evidence. Revised analysis (snapshot v1.2) should further boost scores and possibly trigger a readiness status on roles.
9. *Validation:* Check that 'Evidence Coverage' reaches 100% and new 'target role' might emerge.
10. **Role Gate Check:** For a role requiring, say, $C10 \geq 80$ for "Consulting Manager", simulate a candidate with $C10=50$.
11. *Expected:* Even if aggregated fit is 90%, the UI marks "Consulting Manager – DEVELOPING (C10 gap)".
12. *Validation:* Manually inspect or unit-test role matching logic to catch mandatory gaps.
13. **Edge Case – No Evidence:** Call `GET /api/analysis/unknown-id` . Confirm 404. Also try `/case/unknown-id` to see error handling.
14. **UI Flow Demo:** Record or live-run the UI: start from resume page (or directly show dashboard), click to view competency C09, see evidence cards, then simulate adding case and show dynamic update.

Demo Pitch Scenario:

- *Step 1:* Show baseline dashboard (67 CCI, 49% coverage) and explain meaning.

- *Step 2:* Click through C09 to inspect rationale and evidence. Emphasize evidence tracing.
- *Step 3:* Submit case answer in UI and show spinner.
- *Step 4:* Show updated dashboard (e.g. 72 CCI, 82% coverage, 78% confidence) with annotation "Case added (85% confidence)".
- *Step 5:* Highlight how missing gaps shrank. Mention optional interview step for completeness (if implemented).
- *Conclusion:* "This evidence-first loop ensures decisions are data-backed and reproducible [16†L139-L147] ."

Through these steps, we prove that the system meets the hackathon's metrics and showcases the key innovation of *triangulating evidence from multiple sources*.

8. Top-3 Next Features (with Code Changes)

1. Consulting Case Submission (High Impact):

- *Backend:* Create `backend/app/api/case.py`:

```
from fastapi import APIRouter, HTTPException
from app.evidence.store import get_candidate_evidence,
save_candidate_evidence
from app.scoring.analysis_store import get_saved_analysis, save_analysis
from app.evidence.data_models import EvidenceItem

router = APIRouter(prefix="/case")
@router.post("/{candidate_id}")
def submit_case(candidate_id: str, case: dict):
    prompt = case.get("prompt", "")
    answer = case.get("answer", "")
    if not answer: raise HTTPException(400, "Answer required")
    # Save case answer as evidence
    evidence = get_candidate_evidence(candidate_id) or []
    new_id = f"E-c{len(evidence)+1:03d}"
    ev = EvidenceItem(
        id=new_id, candidate_id=candidate_id,
        source="case", evidence_type="work_sample",
        text=answer, competency_ids=[], confidence=0.85,
        created_at=datetime.now().isoformat(),
        source_version="case-prompt-v1"
    )
    evidence.append(ev)
    save_candidate_evidence(candidate_id, evidence)
    # Rerun analysis with updated evidence
    analysis = _rerun_analysis(candidate_id)
```

```

save_analysis(candidate_id, analysis)
return analysis

```

- Frontend: In `page.tsx`, add a new component:

```

// Pseudocode for Case submission UI
function CaseForm() {
  const [answer, setAnswer] = useState("");
  const handleSubmit = async () => {
    await fetch(`http://127.0.0.1:8000/api/case/candidate-demo-001`, {
      method: "POST", headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ answer })
    });
    window.location.reload(); // refresh dashboard
  };
  return (
    <section>
      <h2>Consulting Case Assessment</h2>
      <p>Prompt: Reduce onboarding time by 50% for a retail bank.</p>
      <textarea value={answer} onChange={e=>setAnswer(e.target.value)} />
      <button onClick={handleSubmit}>Submit Answer</button>
    </section>
  );
}

```

(Integrate into main `Home` component before or after metrics.)

2. AI Interview Module (High Impact):

- Backend: Add `backend/app/api/interview.py`:

```

router = APIRouter(prefix="/interview")
interview_questions = [
  {"id": "Q1", "text": "Describe a time you led change in a team."},
  {"id": "Q2", "text": "How do you prioritize conflicting tasks?"},
  {"id": "Q3", "text": "Tell us about influencing stakeholders."}
]
@router.post("/{candidate_id}")
def submit_interview(candidate_id: str, response: dict):
  qid = response.get("question_id"); ans = response.get("answer", "")
  if not qid or not ans: raise HTTPException(400)
  # Append each answer as evidence item
  evidence = get_candidate_evidence(candidate_id) or []
  new_id = f"E-i{len(evidence)+1:03d}"
  ev = EvidenceItem(
    id=new_id, candidate_id=candidate_id,

```

```

        source="interview", evidence_type="self_report",
        text=ans, competency_ids=[], confidence=0.9, created_at=...,
source_version=qid
    )
    evidence.append(ev)
    save_candidate_evidence(candidate_id, evidence)
    # After last question, rerun analysis
    if qid == "Q3":
        analysis = _rerun_analysis(candidate_id)
        save_analysis(candidate_id, analysis)
        return analysis
    return {"next_question": next_question_text}

```

- *Frontend*: Build a simple interview wizard. Pseudocode:

```

function Interview() {
    const [step, setStep] = useState(0);
    const [answer, setAnswer] = useState("");
    const questions = [ ... ];
    const handleNext = async () => {
        const res = await fetch(.../api/interview/candidate-demo-001`, {
            method: "POST", body: JSON.stringify({ question_id:
questions[step].id, answer })
        });
        const data = await res.json();
        if (step < questions.length-1) {
            setStep(step+1); setAnswer("");
        } else {
            // Final, show updated dashboard
            setAnalysis(data);
            setStep(steps.length);
        }
    };
    // Render current question or dashboard based on step
}

```

- *UI Copy*:
- Questions should map to key competencies (C13 “leadership”, C02 “process thinking”, etc.).
- Use encouraging text (“Feel free to speak to professional experiences.”).

3. Mandatory Gate & Role Readiness (High Impact):

- *Backend/Scoring*: In `match_all_roles`, after matching, add:

```

# Pseudo:
for match in role_matches:

```

```

if match.role_id == "AI_ML_CONSULTANT":
    if any(e.score < 50 for e in evaluations if e.competency_id in
["C10", "C13"]):
        match.readiness = "Developing" # or set match_score lower

```

- *Frontend:* In role-row rendering:

```

<div className={`role-rank ${match.readiness === "Developing" ? "warn" :
""}`}>
    {match.role_rank}
</div>
<div className="role-info">
    <strong>{match.role_name}</strong>
    <span>
        {match.gaps.length === 0 ? "No material gaps" :
        `${match.gaps.length} gap${match.gaps.length>1?"s":""}`}
        {match.readiness === "Developing" && " (Developing fit)"}
    </span>
</div>

```

Add CSS for `.warn` to highlight.

- *Legend:* Possibly add a tooltip: “Developing fit means key competencies need work.”

Each of these features should be developed, tested, and merged before the final demo. They directly target the proof-of-concept criteria from the hackathon: enriched evidence intake (case/interview), transparency (gates/provenance), and a complete user journey.

References: We leveraged design best practices and recent examples to guide our approach. For instance, the principle “Good design is as little design as possible” suggests we keep the UI clean and focused [12†L61-L69]. Modern AI pipeline examples use multi-agent orchestration to handle sub-tasks (e.g. separate agents for resume analysis, matching, and content generation) [11†L29-L37]. And industry guidance on structured AI interviews emphasizes identical questioning and consistent scoring for fairness [16†L139-L147]. These informed our roadmap and will help ensure our METI prototype is robust, fair, and recruiter-friendly.
